

Guide de contribution, onboarding, besoins matériels et estimation des coûts

Document de synthèse rédigé à partir des documents méthodologiques et de veille du projet.

Ce document regroupe trois livrables opérationnels :

- un guide d'intégration et de contribution pour les nouveaux membres ;
- une définition des besoins matériels et logiciels par profil ;
- une estimation budgétaire initiale pour le développement et le déploiement.

Il sert de base de travail pour la mise en route du projet et pour les arbitrages de moyens.

Contenu

Projet : Flowly

Version : 1.0

Date : Mars 2026

Source : Documents internes

Important : la partie "coûts" combine des informations explicites des documents (outils gratuits, comptes de publication mobile) et des estimations opérationnelles formulées comme hypothèses, lorsque les sources ne donnent pas de prix d'infrastructure ou de matériel.

1. Objet du document

Le présent document consolide les règles de contribution, les prérequis d'intégration et les besoins de moyens nécessaires au projet Flowly. Il s'adresse en priorité à un nouveau membre d'équipe, mais peut aussi servir de référence pour cadrer le setup initial, l'organisation du travail et le budget minimal de lancement.

Sources principales mobilisées : Normes & Méthodologie Flowly, veille frontend, veille backend, veille mobile, veille DevOps.

2. Synthèse projet et architecture cible

Flowly est une application de productivité gamifiée, synchronisée entre desktop et mobile, avec monde 3D, sessions de focus, analytics et fonctions sociales. Le socle cible ressort clairement des documents : frontend desktop en Electron + React + TypeScript + Three.js, backend Node.js 20 / NestJS / PostgreSQL / Redis / Prisma, mobile en SwiftUI pour iOS et Flutter pour Android, et approche DevOps combinant Docker en développement puis Kubernetes en production.

Volet	Technologies clés	Finalité	Implication onboarding
Desktop	Electron, React, TypeScript, Three.js, Zustand	UI desktop, monde 3D, intégrations système via modules natifs	Installer la base front et comprendre l'architecture par domaines.
Backend	Node.js 20, NestJS, REST, JWT/OAuth, PostgreSQL, Redis, Prisma	API, auth, synchronisation et logique métier	Configurer Node, base locale, variables d'environnement et tests.
Mobile	SwiftUI (iOS), Flutter (Android), MVVM, WebView, notifications	Fonctions natives, publication stores, cohérence UX	Choisir le parcours iOS ou Android selon le rôle et les accès.
DevOps	Docker, GitLab CI ou équivalent, Terraform, Ansible, Trivy, SonarQube, Prometheus, Grafana	Build, déploiement, sécurité et supervision	Comprendre la chaîne CI/CD, les secrets et les environnements.

Point de vigilance : les documents sont cohérents sur la stack applicative, mais pas totalement alignés sur l'outillage CI/CD. La méthodologie cite GitHub pour le dépôt, les PR et la CI/CD, tandis que la veille DevOps préfère GitLab CI pour centraliser le projet. Ce point doit être arbitré avant de figer le guide d'onboarding.

3. Guide de contribution et onboarding

3.1 Principes de fonctionnement

Le projet suit Scrum avec des sprints de deux semaines, un daily quotidien, une sprint planning en début de sprint, une review et une rétrospective en fin de sprint, ainsi qu'un refinement intermédiaire. Toute nouvelle contribution doit s'inscrire dans cette cadence et dans la logique d'amélioration continue décrite dans les documents.

3.2 Accès à obtenir dès l'arrivée

Outil / accès	Usage	Priorité	Action attendue du nouveau membre
ClickUp	Backlog, sprints, tickets, roadmap, docs	Critique	Rejoindre l'espace, lire les tickets ouverts et comprendre le sprint en cours.
Discord	Communication, daily, pair programming, annonces	Critique	Rejoindre les canaux texte et les salons vocaux du projet.
GitHub / repo	Versioning, branches, PR, code review, CI/CD	Critique	Cloner le dépôt, configurer SSH et vérifier les droits de push sur branches dédiées.
ClickUp Docs	Wiki technique, CR de réunion, templates	Élevée	Lire les conventions, l'architecture et les comptes-rendus essentiels.
Figma	Maquettes, prototypes, design system	Élevée	Consulter les composants et parcours si la mission touche à l'UI/UX.

Canaux Discord à rejoindre : #général, #dev, #design, #bugs, #daily-standup, #liens-utiles, ainsi que les salons Daily, Cérémonies, Pair Programming et Chill.

3.3 Lecture minimale obligatoire avant première contribution

- les normes de code et les conventions de nommage ;
- la Definition of Done et les seuils de couverture de tests ;
- la stratégie de branches et les conventions de commits ;
- la structure de la stack sur le périmètre concerné (front, back, mobile ou DevOps).

3.4 Workflow de contribution attendu

1. Créer une branche dédiée depuis develop selon le format feature/FLOW-[ticket]-description-courte, ou bugfix/hotfix selon le besoin.
2. Implémenter le ticket en respectant ESLint, Prettier et les conventions de structure du code.
3. Écrire ou compléter les tests associés, puis vérifier que la CI locale ou distante passe.
4. Ouvrir une Pull Request, demander une review et corriger les retours avant merge.
5. Mettre à jour la documentation si la fonctionnalité, l'architecture ou le setup ont évolué.

3.5 Standards à appliquer systématiquement

Sujet	Règle	Application concrète
Nommage	camelCase, PascalCase, SCREAMING_SNAKE_CASE, kebab-case selon le type	Permet d'identifier immédiatement variables, composants, constantes ou utilitaires.

Sujet	Règle	Application concrète
Formatage	2 espaces, 100 caractères max, point-virgule obligatoire	Réduction des écarts de style et lecture homogène entre membres.
Commits	Conventional Commits	Ex. feat(timer), fix(auth), docs(readme), test(api).
Branches	main / develop / feature bugfix hotfix	Travail isolé par ticket et intégration contrôlée.
Qualité	ESLint + Prettier obligatoires	Contrôle automatisé avant review et avant merge.

3.6 Definition of Done et qualité minimale

Un ticket n'est terminé que si le code compile, si aucune erreur ESLint/Prettier ne subsiste, si les tests sont écrits et passants, si une review a été approuvée, si la documentation a été mise à jour au besoin et si les critères d'acceptation de la User Story sont validés. Les objectifs de couverture sont les suivants : 70 % minimum et 85 % cible pour les tests unitaires, 80 % minimum et 95 % cible pour les tests d'intégration API, et 100 % des parcours critiques en E2E.

3.7 Parcours d'intégration recommandé sur 5 jours

Jour	Objectif	Actions	Livrable attendu
J1	Prise d'accès	Configurer ClickUp, Discord, dépôt Git et lecture des documents de base.	Membre autonome sur les outils.
J2	Setup local	Installer la stack de son périmètre, récupérer les variables locales, lancer l'application.	Environnement fonctionnel.
J3	Compréhension du flux	Observer une daily, lire le sprint en cours et les conventions de contribution.	Capacité à suivre le travail de l'équipe.
J4	Premier ticket encadré	Prendre un ticket simple ou documentaire, coder et tester avec un référent.	Première PR ouverte.
J5	Boucle de review	Traiter les retours, merger si validé, documenter ce qui a été appris.	Contribution intégrée et onboarding validé.

4. Besoins matériels et logiciels

Les documents décrivent précisément les besoins logiciels et technologiques du projet, mais ne fixent pas une configuration matérielle détaillée poste par poste. La grille ci-dessous formalise donc des besoins matériels recommandés pour développer dans de bonnes conditions, à partir de la stack retenue et des contraintes techniques du projet.

4.1 Logiciels et environnements à prévoir

- Gestion de projet et collaboration : ClickUp, Discord, ClickUp Docs, Figma.
- Frontend desktop : Node.js, package manager, Electron, React, TypeScript, Three.js, React Three Fiber, Framer Motion, Zustand.
- Backend : Node.js 20 LTS, NestJS, PostgreSQL, Redis, Prisma, Jest, Supertest, Swagger/OpenAPI.
- Mobile : Xcode / TestFlight / App Store Connect pour iOS ; Android Studio / Google Play Console pour Android ; Flutter et Dart pour Android si cette piste est retenue.
- DevOps : Docker, CI/CD, Terraform, Ansible, Trivy, SonarQube, Prometheus, Grafana, gestion de secrets.

4.2 Recommandations matérielles par profil

Profil	CPU / usage	RAM conseillée	Stockage	Remarques
Développeur polyvalent	4 à 8 coeurs	16 Go min. / 32 Go confort	SSD 512 Go	Base recommandée pour Node, Docker et IDE modernes.
Frontend / 3D	CPU récent + GPU correcte	16 à 32 Go	SSD 512 Go à 1 To	Electron + rendu 3D et assets graphiques demandent plus de marge.
Backend / DevOps	CPU multi-coeurs	16 à 32 Go	SSD 512 Go	Les conteneurs, la base locale, Redis et les tests peuvent être gourmands.
iOS	Poste macOS requis	16 Go recommandés	SSD 512 Go	Xcode, TestFlight et App Store Connect impliquent une machine Apple dédiée au build.
Android	CPU récent	16 Go recommandés	SSD 512 Go	Android Studio + émulateurs nécessitent de la RAM et du stockage.

4.3 Matériel partagé conseillé

Équipement	Pourquoi le prévoir	Niveau de priorité
1 iPhone de test	Validation des parcours natifs iOS et des permissions système.	Élevée
1 smartphone Android de test	Validation du monitoring d'applications, des notifications et du background.	Élevée
Écran secondaire	Confort de travail sur Figma, code et monitoring.	Moyenne
Compte cloud / VM de staging	Tester le déploiement hors machine locale et la reproductibilité.	Élevée

4.4 Ce que les documents permettent et ne permettent pas

Les sources permettent de justifier le besoin d'un poste de développement solide, d'un poste Apple pour iOS, d'au moins un appareil Android et iPhone de test, ainsi que des outils Docker / CI / monitoring. En revanche, elles ne fournissent pas de catalogue d'achat ni de prix matériels ; un chiffrage d'acquisition poste par poste doit donc être traité comme un devis séparé.

5. Estimation des coûts potentiels

La matrice ci-dessous distingue les coûts directement identifiables dans les documents et les estimations formulées à titre opérationnel. Elle vise à cadrer un budget de démarrage, pas à remplacer un chiffrage fournisseur détaillé.

Poste	Base documentaire / hypothèse	Estimation	Commentaire
Outils de gestion et design	Méthodologie : ClickUp Free, Figma Free, Discord, GitHub	0 EUR en base	Le projet peut démarrer sans surcoût logiciel immédiat sur ces briques.
Publication iOS	Veille mobile : compte Apple Developer	99 USD / an	Coût explicite de publication et de distribution iOS.
Publication Android	Veille mobile : compte Google Play Console	25 USD une fois	Coût d'ouverture du compte développeur Android.
Développement local	Docker en développement + outils gratuits	0 EUR a quelques EUR / mois	Peut rester quasi nul si tout tourne en local et sur plans gratuits.
Staging / MVP simple	Hypothèse : 1 environnement léger pour API + base + cache	20 a 80 EUR / mois	Fourchette indicative pour un hébergement basique à faible trafic.
Production légère	Hypothèse : hébergement dédié plus stable, sauvegardes, monitoring	80 a 250 EUR / mois	À ajuster selon trafic, stockage, sauvegardes et niveau de disponibilité.
Cible Kubernetes	Hypothèse : plusieurs noeuds + supervision + stockage	150 EUR / mois et plus	Le coût augmente nettement si la cible Kubernetes est adoptée tôt.

5.1 Lecture budgétaire rapide

Scénario	Ce qu'il couvre	Coût de lancement	Coût récurrent
Prototype académique	Travail local, outils gratuits, pas ou peu d'hébergement	124 USD hors matériel	0 a 20 EUR / mois
MVP démontrable	Publication mobile + petit staging / démo	124 USD hors matériel	20 a 80 EUR / mois
Pré-prod légère	Déploiement plus fiable, monitoring minimal, sauvegardes	124 USD hors matériel	80 a 250 EUR / mois

5.2 Coût majeur non chiffrable à ce stade

Le poste de coût le plus important reste le temps de développement. Les documents montrent que la stratégie mobile repose sur deux codebases distinctes (SwiftUI pour iOS et Flutter pour Android). Sans estimation de charge, taille d'équipe, durée de projet et taux journalier, il n'est pas possible de chiffrer honnêtement ce coût humain.

6. Recommandations finales

- Nommer une source de vérité unique pour le dépôt et la CI/CD (GitHub ou GitLab) avant l'arrivée de nouveaux contributeurs.
- Formaliser un README d'installation par périmètre : desktop, backend, iOS, Android, DevOps.
- Prévoir dès le début un kit matériel minimal partagé : au moins un Mac iOS, un appareil Android et un environnement de staging.
- Conserver les outils gratuits au lancement, puis n'ouvrir des dépenses d'infrastructure que lorsque la démonstration ou la préproduction l'exigent réellement.
- Mettre en avant un premier ticket encadré dans le parcours d'onboarding pour sécuriser l'intégration du nouveau membre.

7. Sources documentaires utilisées

- Final_Flowly_Normes_Methodologie.pdf
- veille_techno_front.pdf
- veille_backend_final.pdf
- Veille technos Mobile-2026031211351774.pdf
- Veille Techno DevOps Flowly.pdf